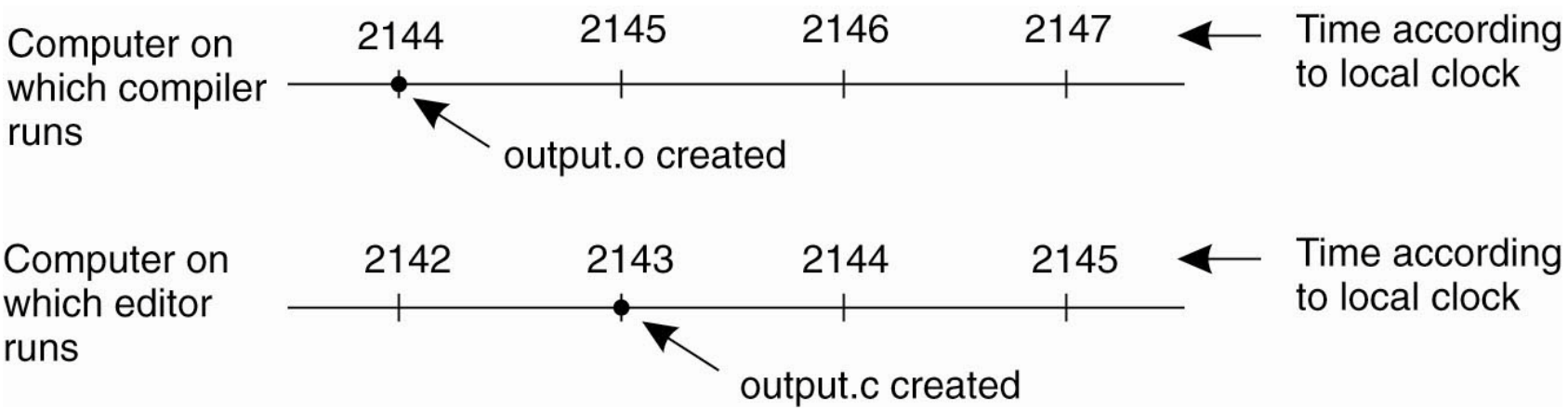
Distributed Systems
Synchronization

Pedro García López

pgarcia@etse.urv.es/

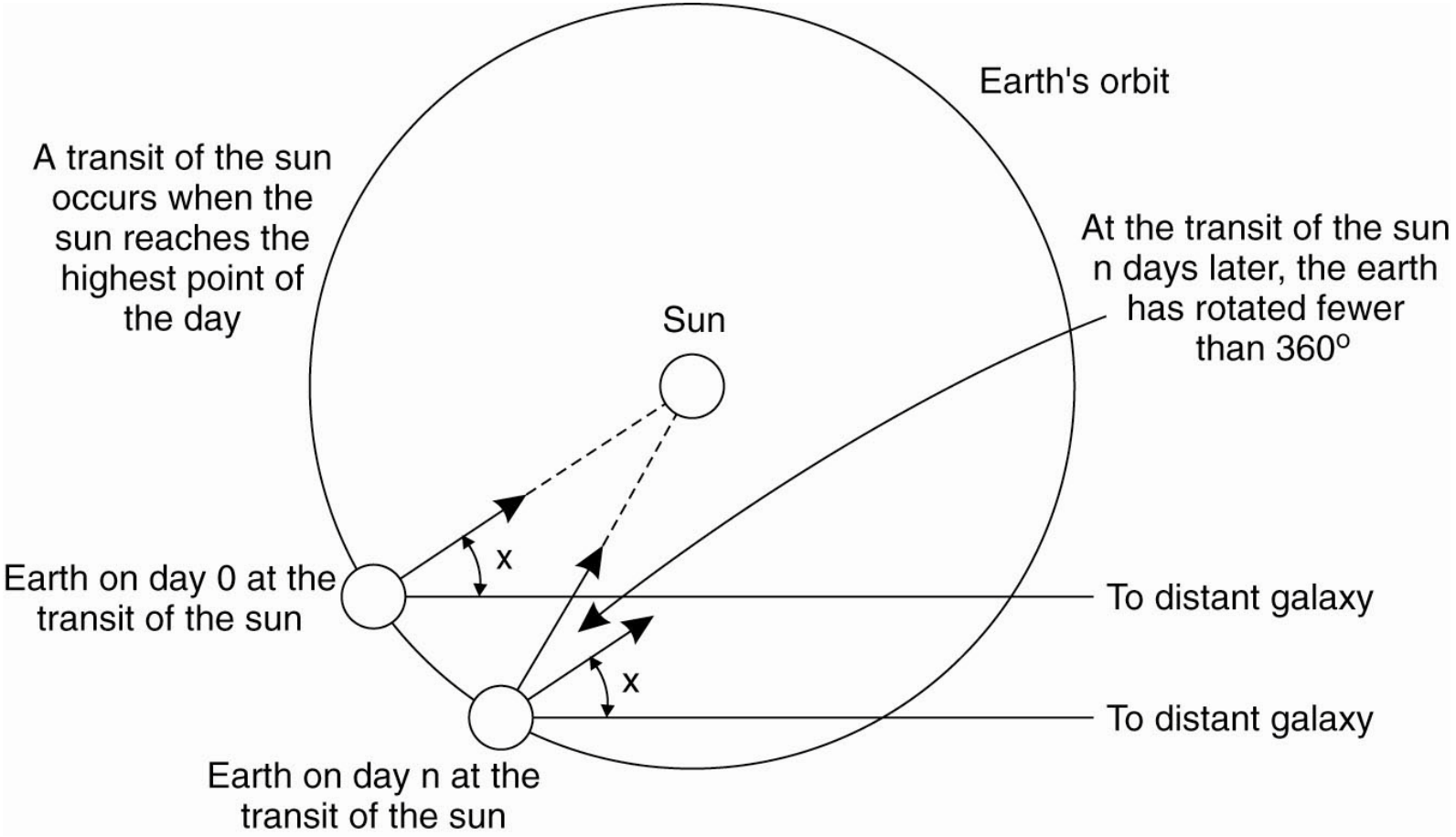
Copyright

Clock Synchronization



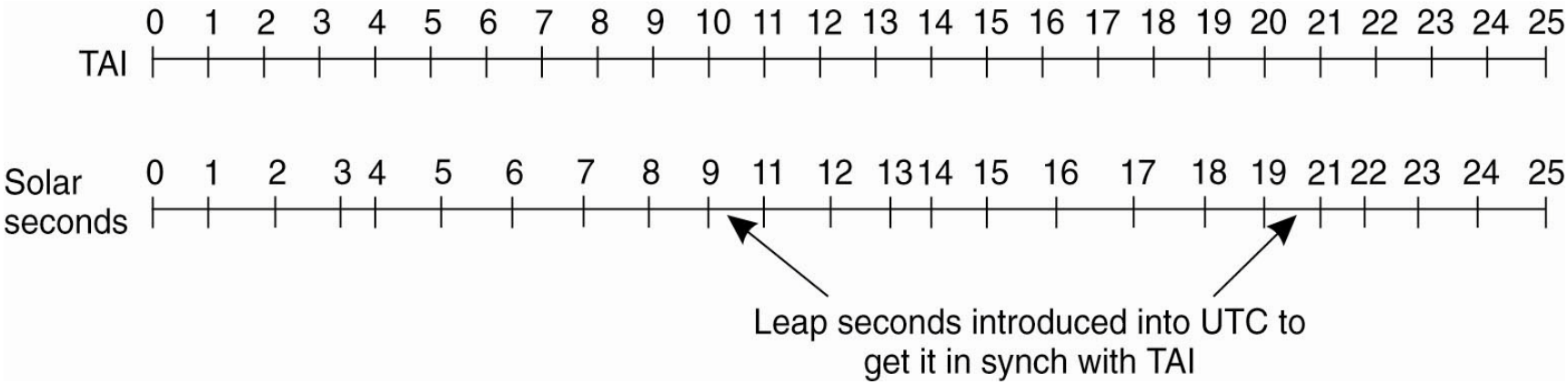
- When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

Physical Clocks (1)



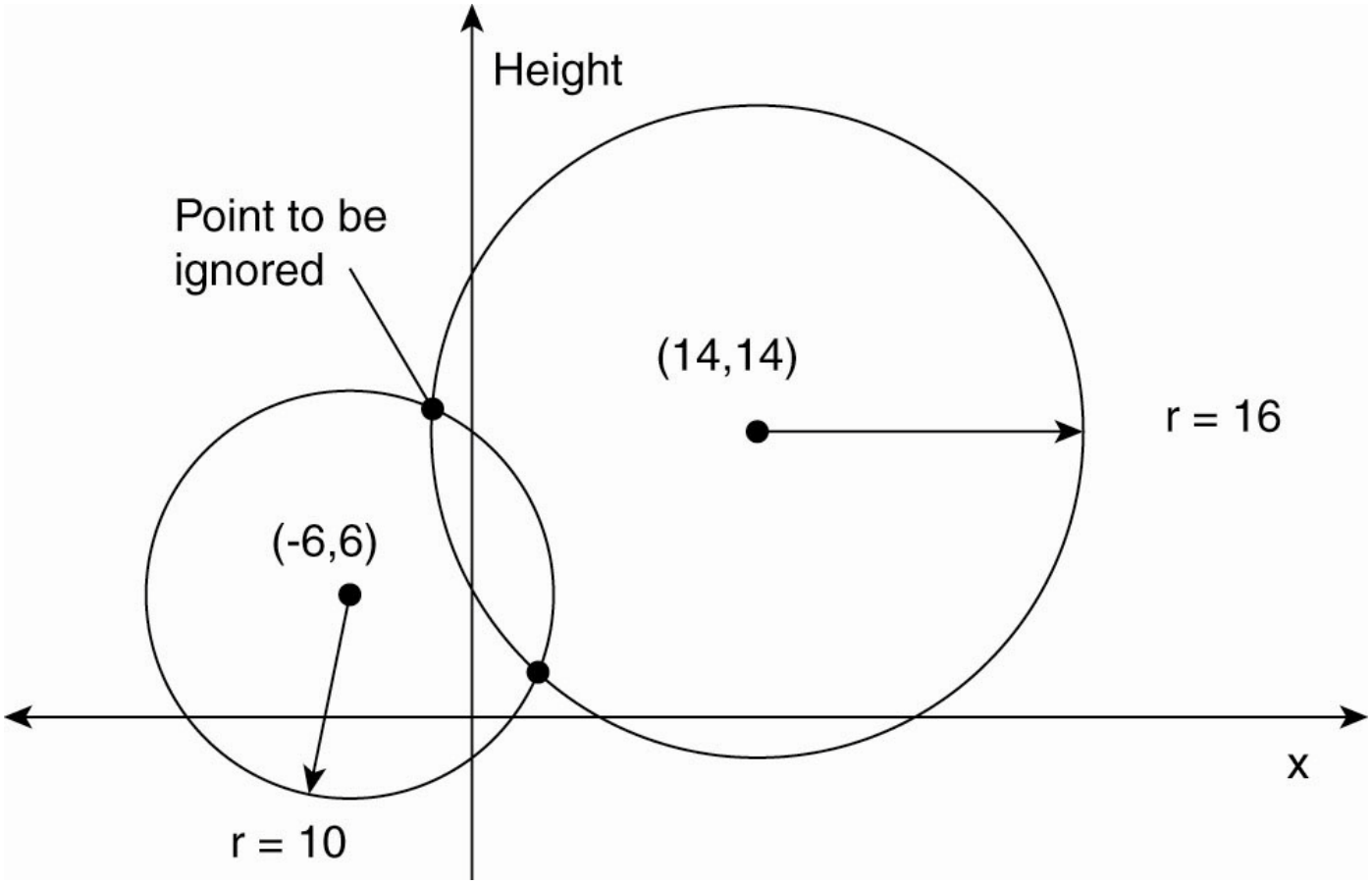
- Computation of the mean solar day.

Physical Clocks (2)



- TAI seconds are of constant length, unlike solar seconds. Leap seconds are introduced when necessary to keep in phase with the sun.

Global Positioning System (1)



- Computing a position in a two-dimensional space.

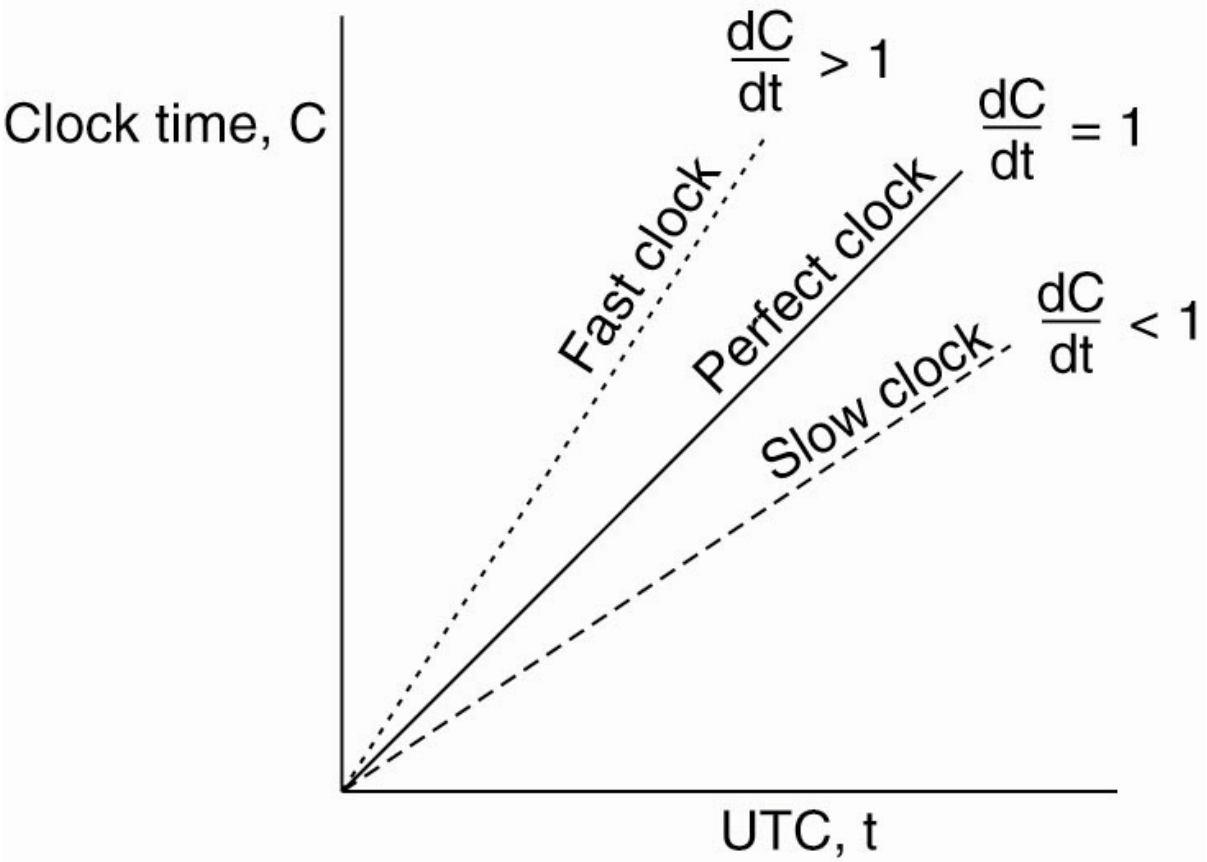
Global Positioning System (2)

- Real world facts that complicate GPS

1. It takes a while before data on a satellite's position reaches the receiver.
2. The receiver's clock is generally not in synch with that of a satellite.

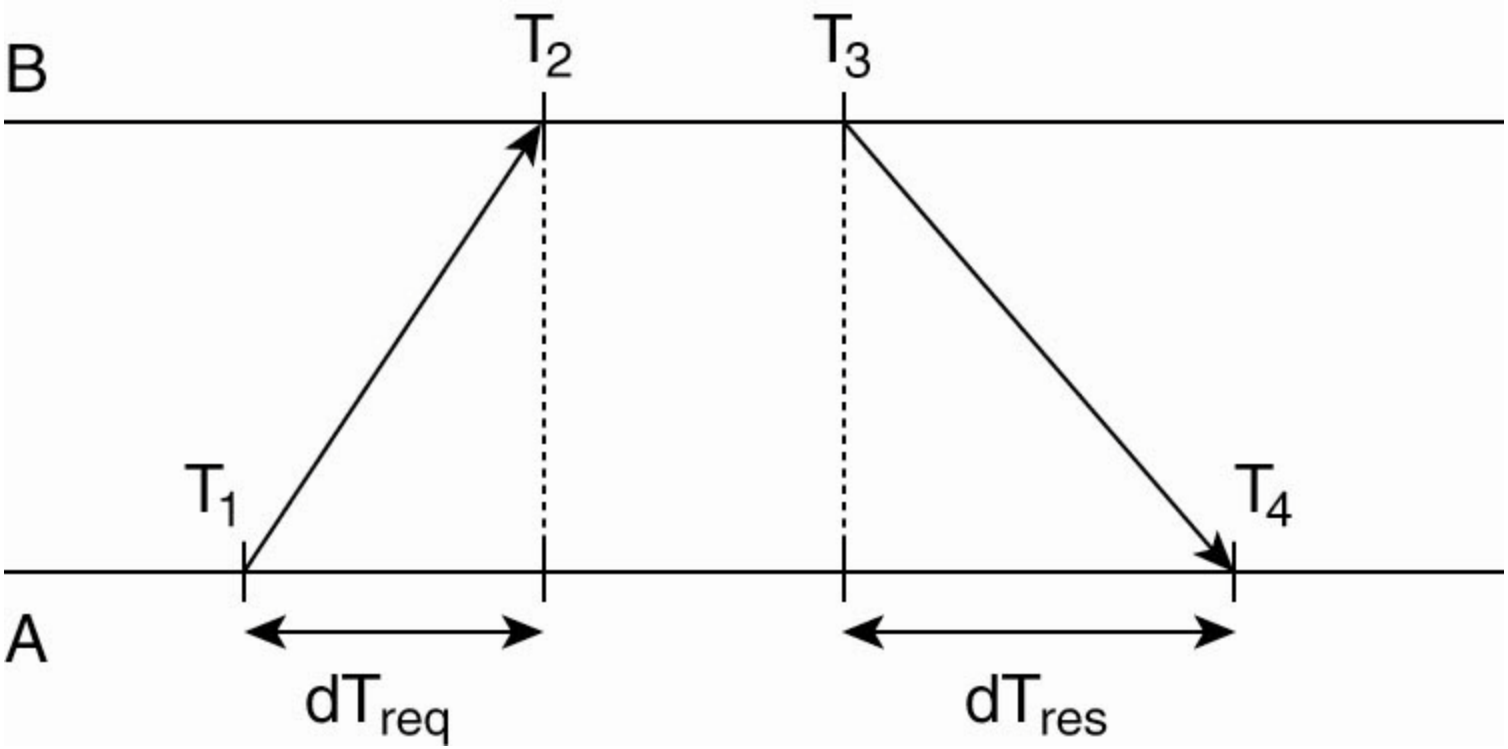
Clock Synchronization Algorithms

- The relation between clock time and UTC when clocks tick at different rates.



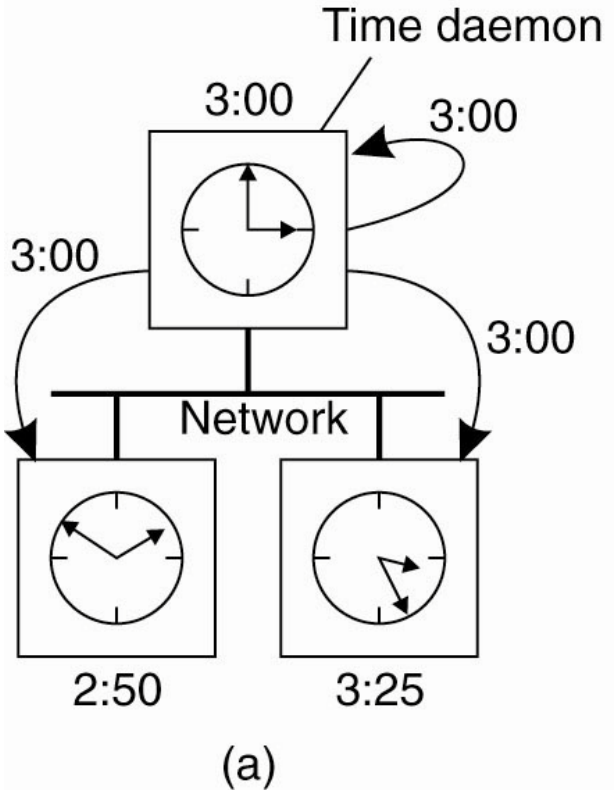
Network Time Protocol

- Getting the current time from a time server.



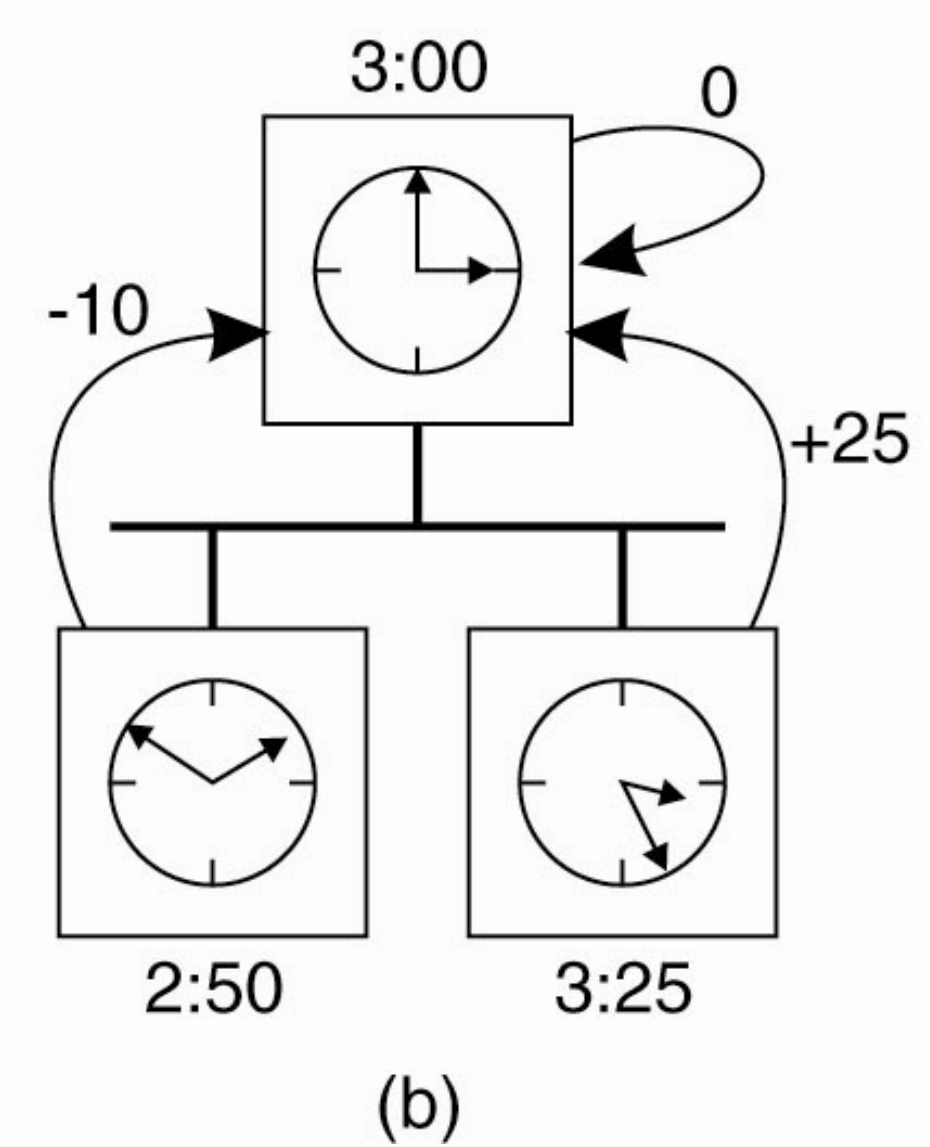
The Berkeley Algorithm (1)

- (a) The time daemon asks all the other machines for their clock values.



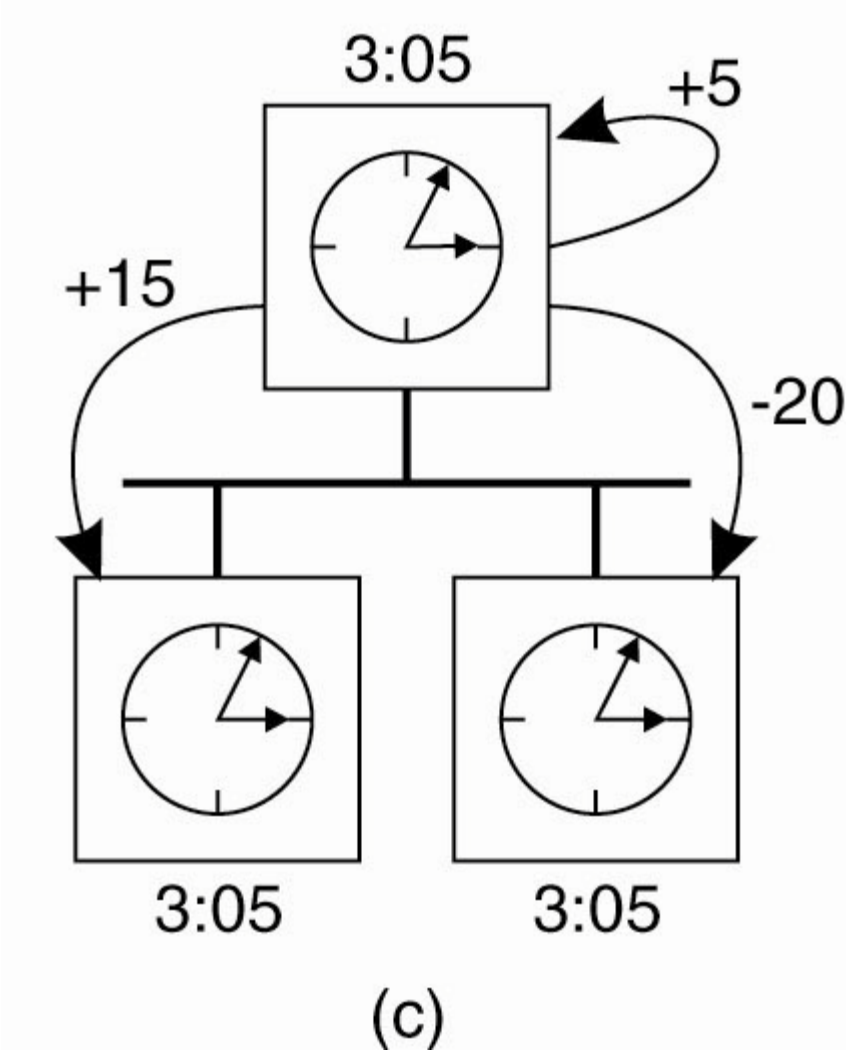
The Berkeley Algorithm (2)

- (b) The machines answer.



The Berkeley Algorithm (3)

- (c) The time daemon tells everyone how to adjust their clock.

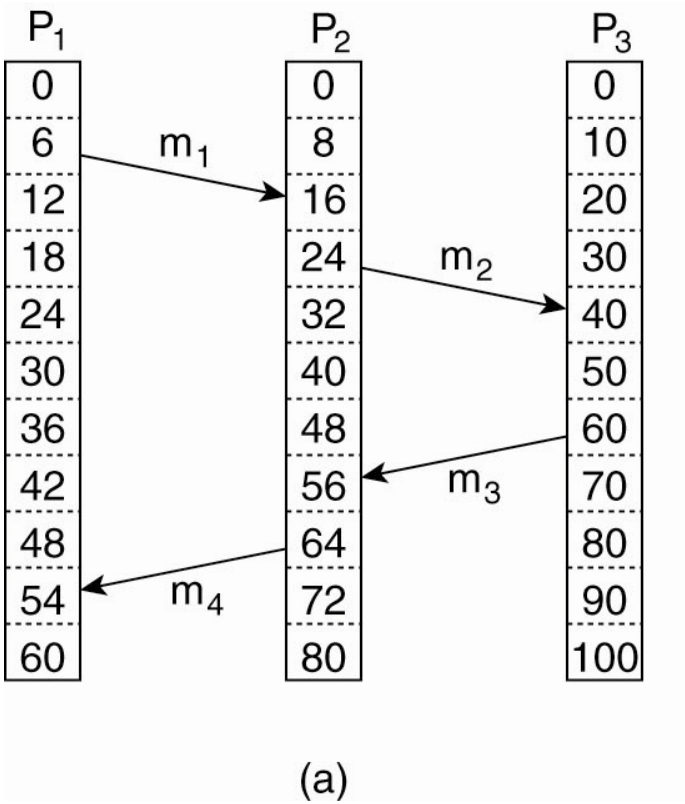


Lamport's Logical Clocks

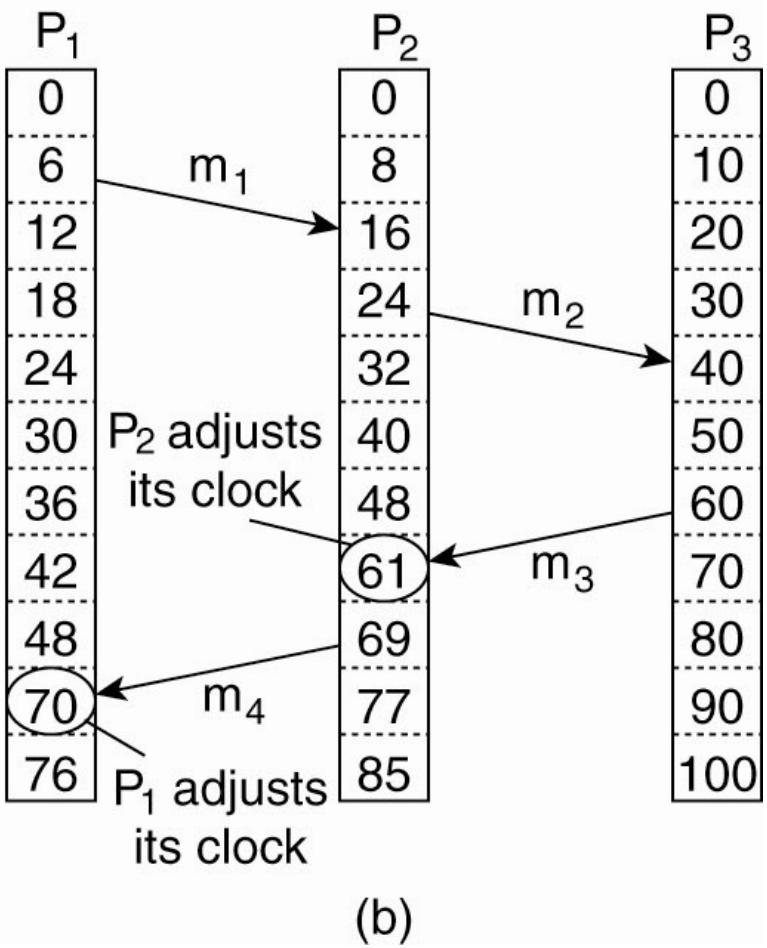
- The "happens-before" relation $\rightarrow$ can be observed directly in two situations:
- If *a* and *b* are events in the same process, and *a* occurs before *b*, then $a \rightarrow b$ is true.
- If a is the event of a message being sent by one process, and *b* is the event of the message being received by another process, then $a \rightarrow b$

Lamport's Logical Clocks

- (a) Three processes, each with its own clock. The clocks run at different rates.

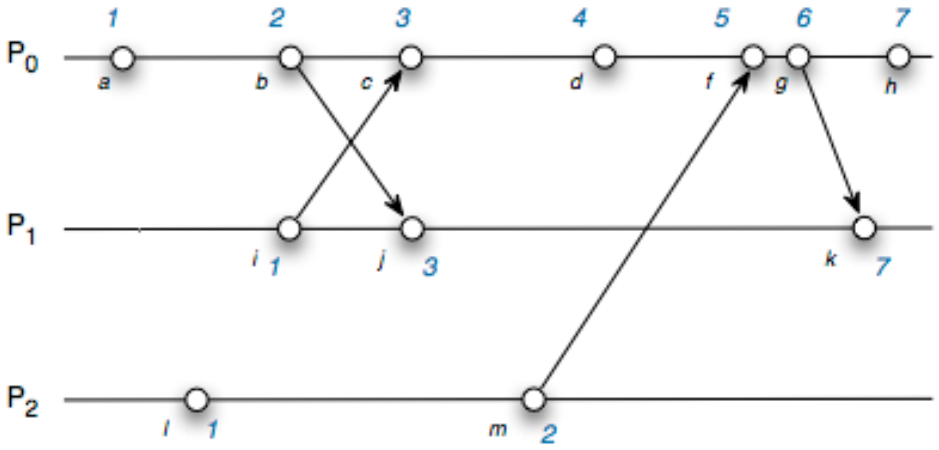


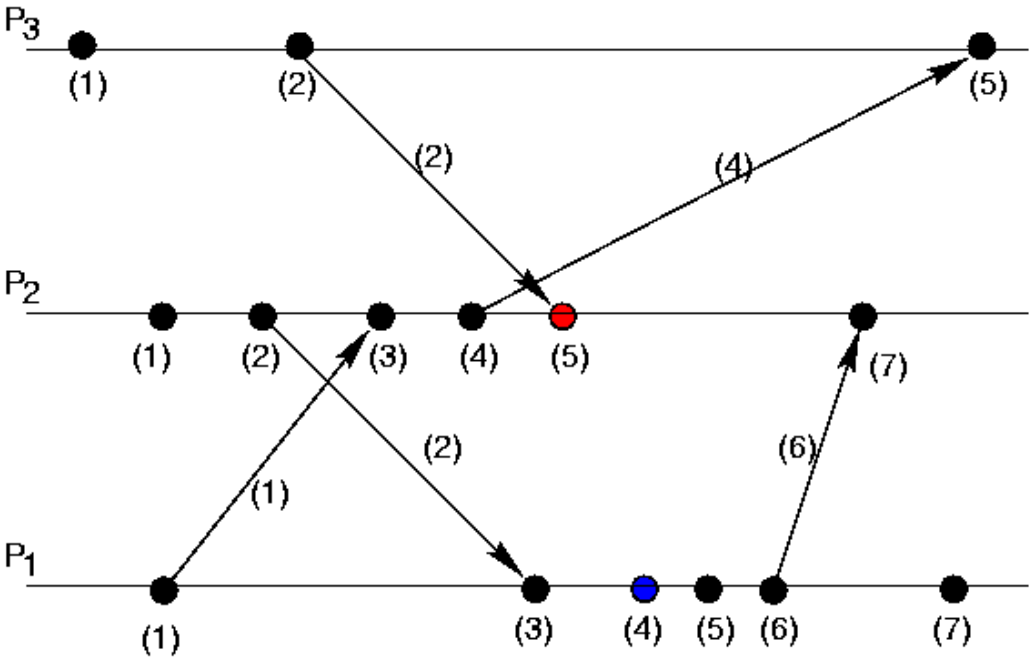
Lamport's Logical Clocks



- (b) Lamport's algorithm corrects the clocks.

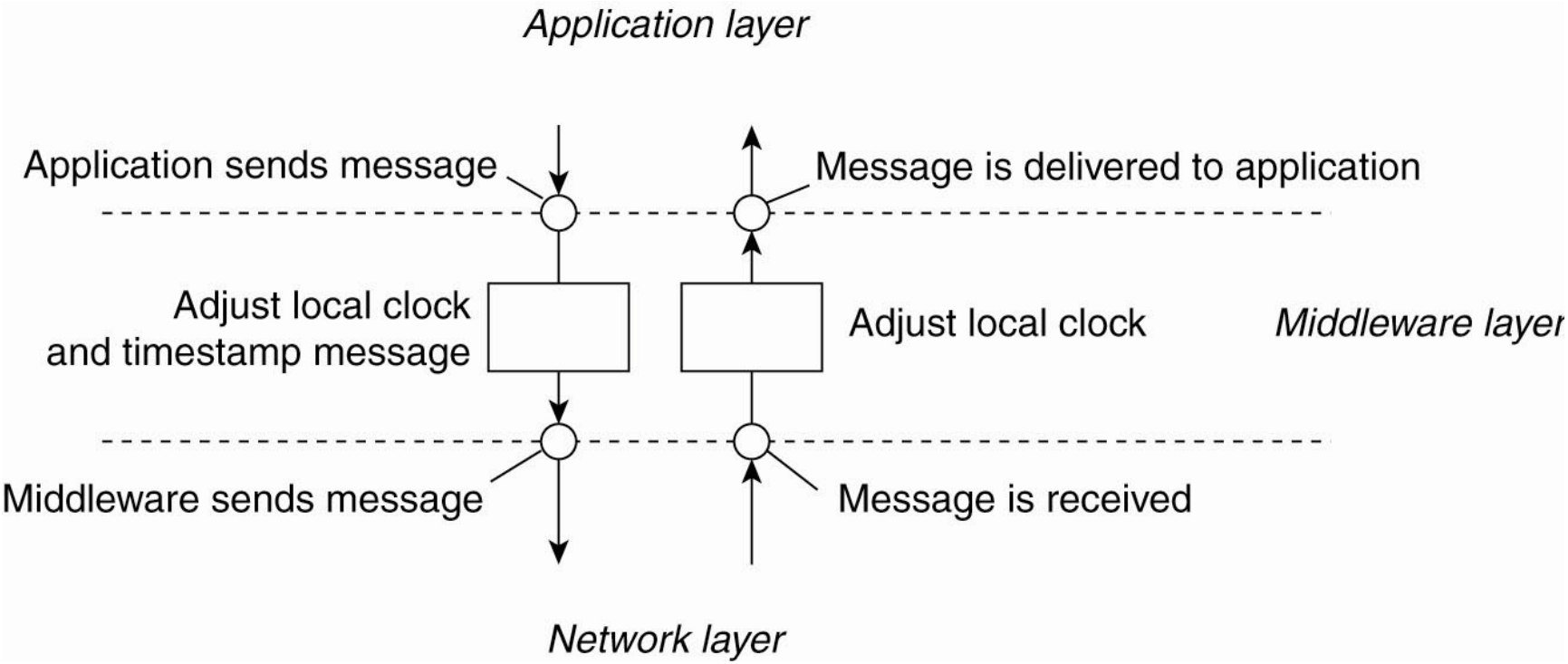
Lamport's Logical Clocks





Lamport's Logical Clocks

- The positioning of Lamport's logical clocks in distributed systems.

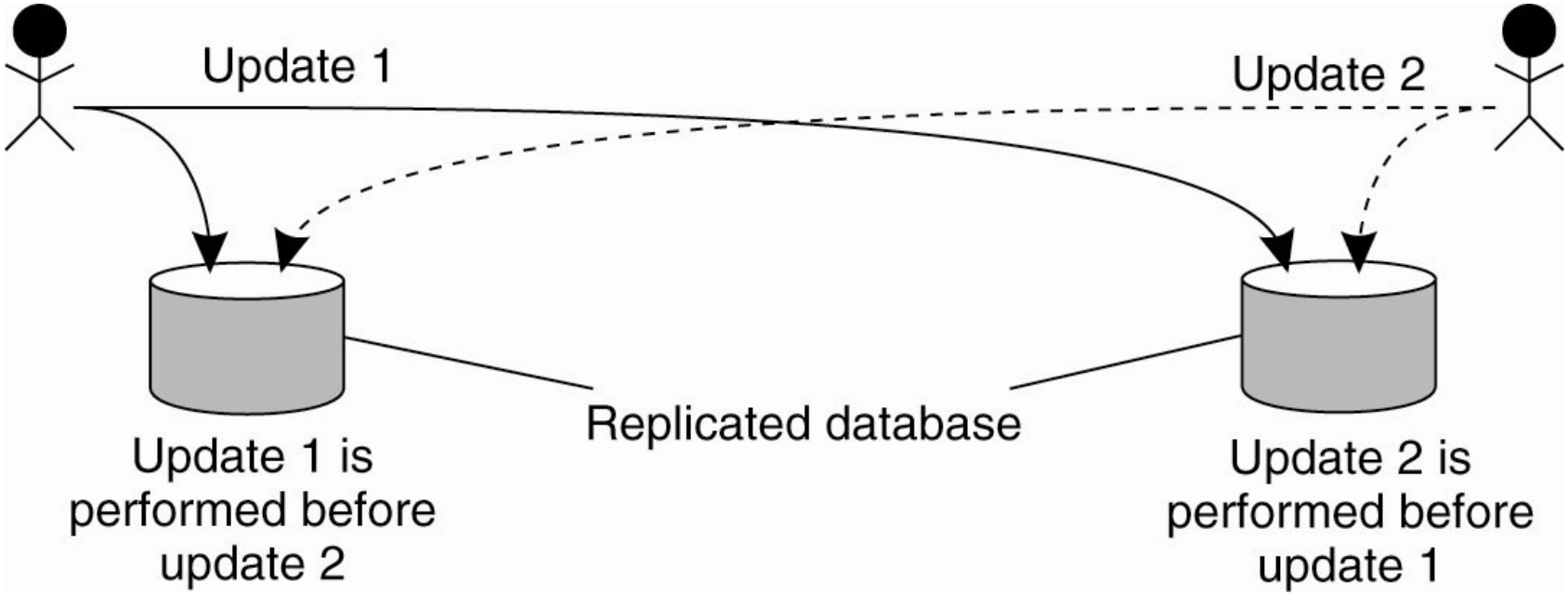


Lamport's Logical Clocks

- Updating counter C_i for process P_i

1. Before executing an event P_i executes
   $C_i \leftarrow C_i + 1$.
2. When process P_i sends a message m to P_j, it sets *m*'s timestamp *ts (m)* equal to C_i after having executed the previous step.
3. Upon the receipt of a message *m*, process P_j adjusts its own local counter as
   $C_j \leftarrow \max\{C_j , ts\ (m)\}$, after which it then executes the first step and delivers the message to the application.

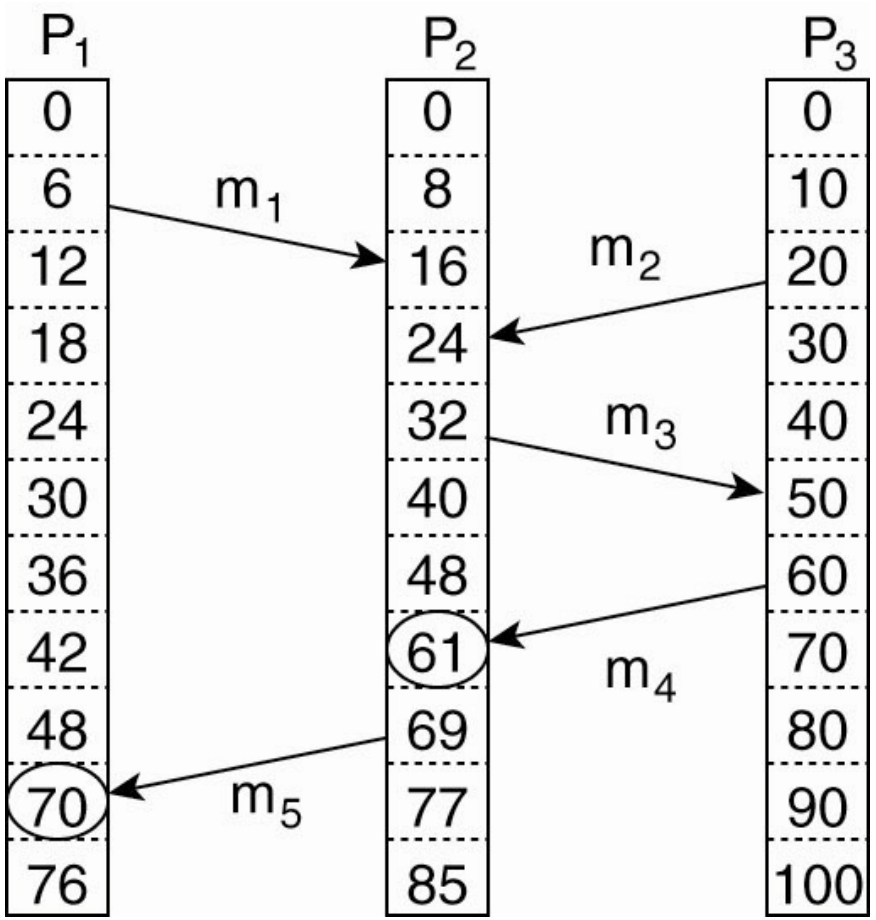
Example: Totally Ordered Multicasting



- Updating a replicated database and leaving it in an inconsistent state.

Vector Clocks (1)

- Concurrent message transmission using logical clocks.



Vector Clocks (2)

- Vector clocks are constructed by letting each process P_i maintain a vector VC_i with the following two properties:

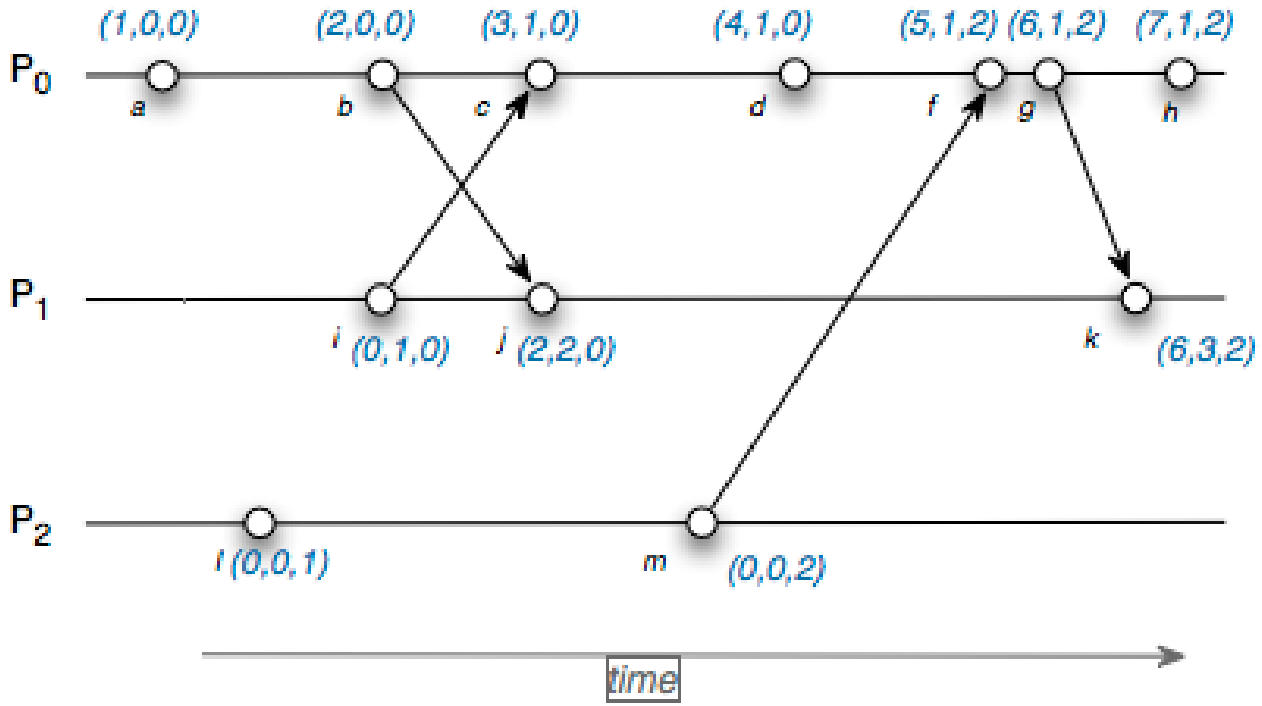
1. VC_i [i] is the number of events that have occurred so far at P_i. In other words, VC_i [i] is the local logical clock at process P_i .
2. If VC_i [j] = k then P_i knows that k events have occurred at P_j. It is thus P_i's knowledge of the local time at P_j .

Vector Clocks (3)

- Steps carried out to accomplish property 2 of previous slide:

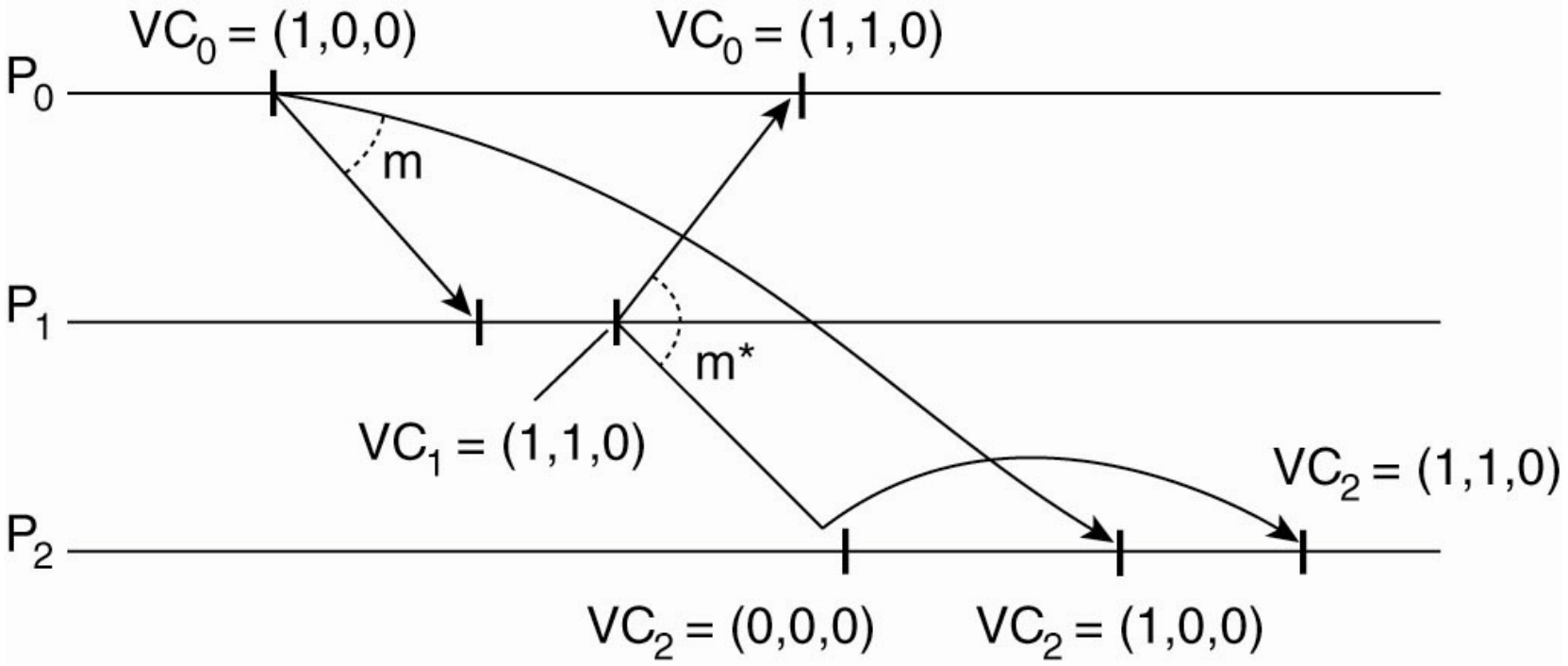
1. Before executing an event P_i executes $VC_i[\,i\,] \leftarrow VC_i[\,i\,] + 1$.
2. When process P_i sends a message m to P_j, it sets *m*'s (vector) timestamp *ts (m)* equal to VC_i after having executed the previous step.
3. Upon the receipt of a message m, process P_j adjusts its own vector by setting $VC_j[k\,] \leftarrow \max\{VC_j[k\,], ts\,(m)[k\,]\}$ for each *k*, after which it executes the first step and delivers the message to the application.

Vector clocks

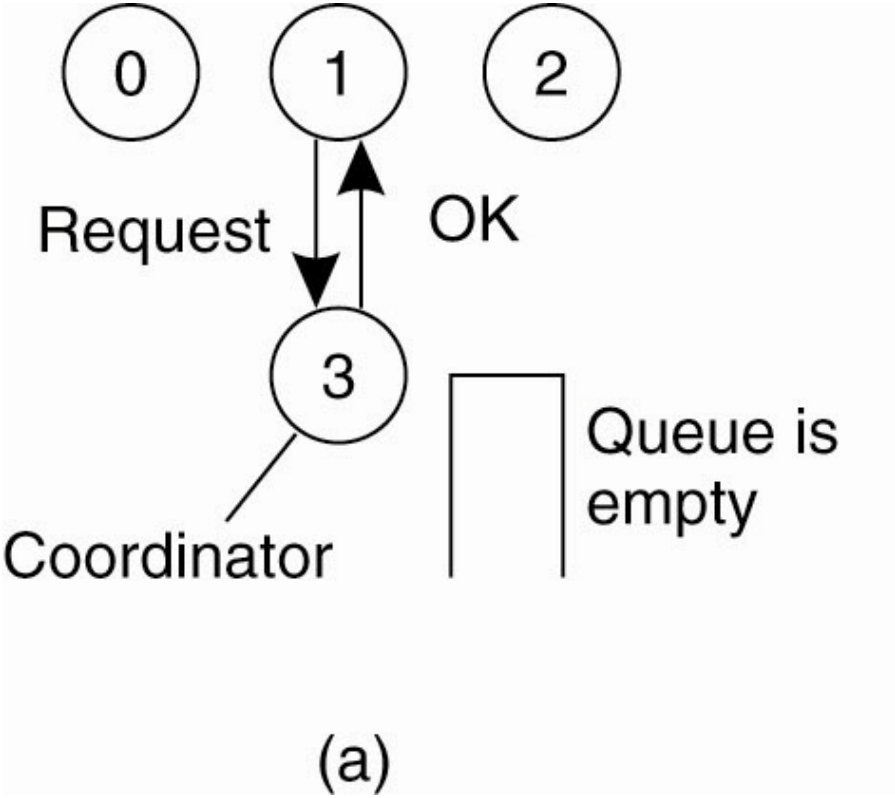


$$x \| y \Leftrightarrow \begin{cases} VC_x[p_x] \geq VC_y[p_x] \quad \text{and} \\ VC_x[p_y] \leq VC_y[p_y], \\ \text{or} \\ VC_x[p_x] \leq VC_y[p_x] \quad \text{and} \\ VC_x[p_y] \geq VC_y[p_y]. \end{cases}$$

Enforcing Causal Communication

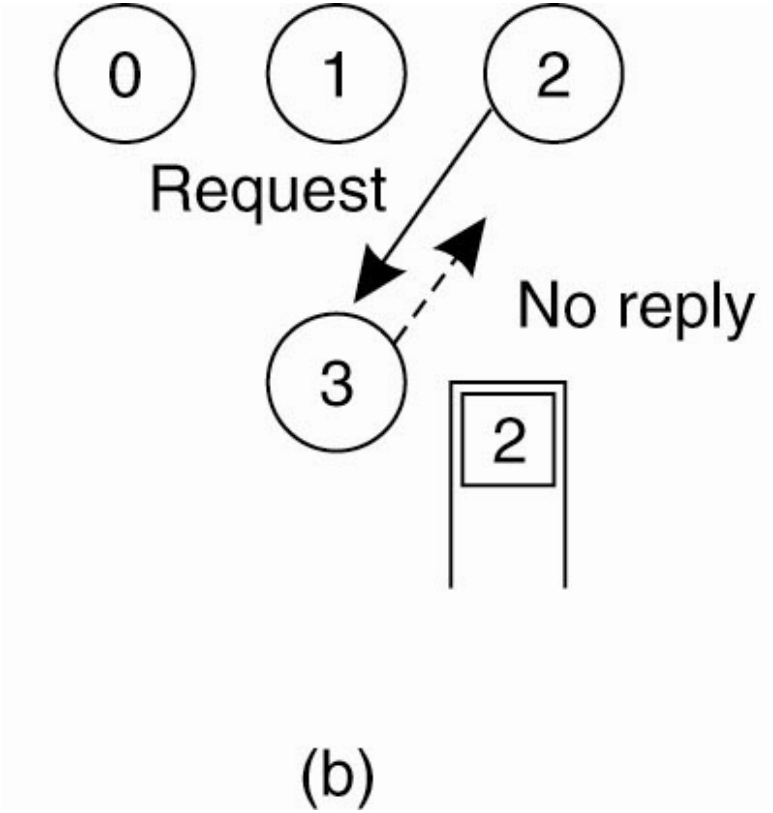


Mutual Exclusion
A Centralized Algorithm (1)



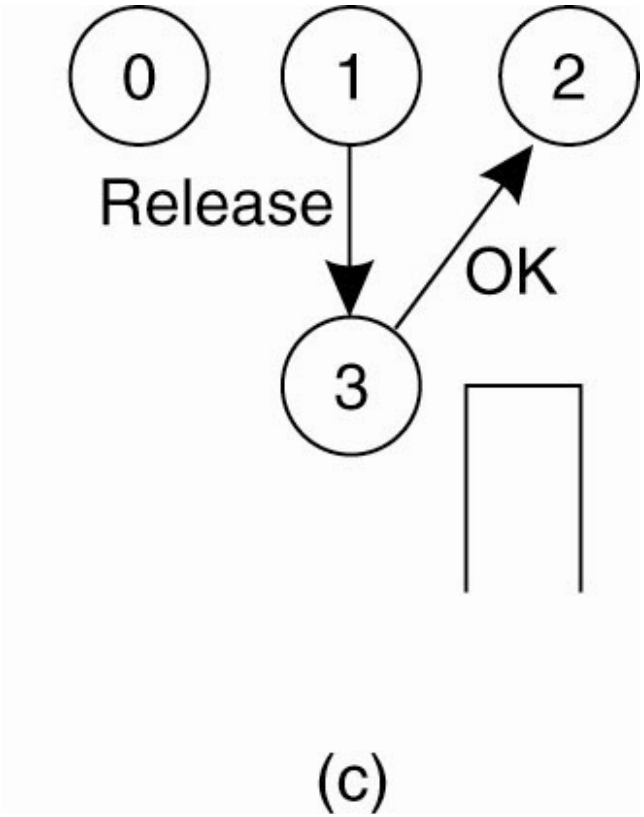
- (a) Process 1 asks the coordinator for permission to access a hared resource. Permission is granted.

Mutual Exclusion
A Centralized Algorithm (2)



- (b) Process 2 then asks permission to access the same resource. The coordinator does not reply.

Mutual Exclusion
A Centralized Algorithm (3)



(c)

- (c) When process 1 releases the resource, it tells the coordinator, which then replies to 2.

A Distributed Algorithm (1)

- Three different cases:

1. If the receiver is not accessing the resource and does not want to access it, it sends back an OK message to the sender.
2. If the receiver already has access to the resource, it simply does not reply. Instead, it queues the request.
3. If the receiver wants to access the resource as well but has not yet done so, it compares the timestamp of the incoming message with the one contained in the message that it has sent everyone. The lowest one wins.

Ricart & Agrawala

On initialization

state := RELEASED;

To enter the section

state := WANTED;

Multicast *request* to all processes;

T := request's timestamp;

Wait until (number of replies received = (*N* – 1));

state := HELD;

request processing deferred here

On receipt of a request $<T_i, p_i>$ *at* p_j $(i \neq j)$

if (*state* = HELD or (*state* = WANTED *and* $(T, p_j) < (T_i, p_i)$))

then

queue *request* from p_i without replying;

else

reply immediately to p_i;

end if

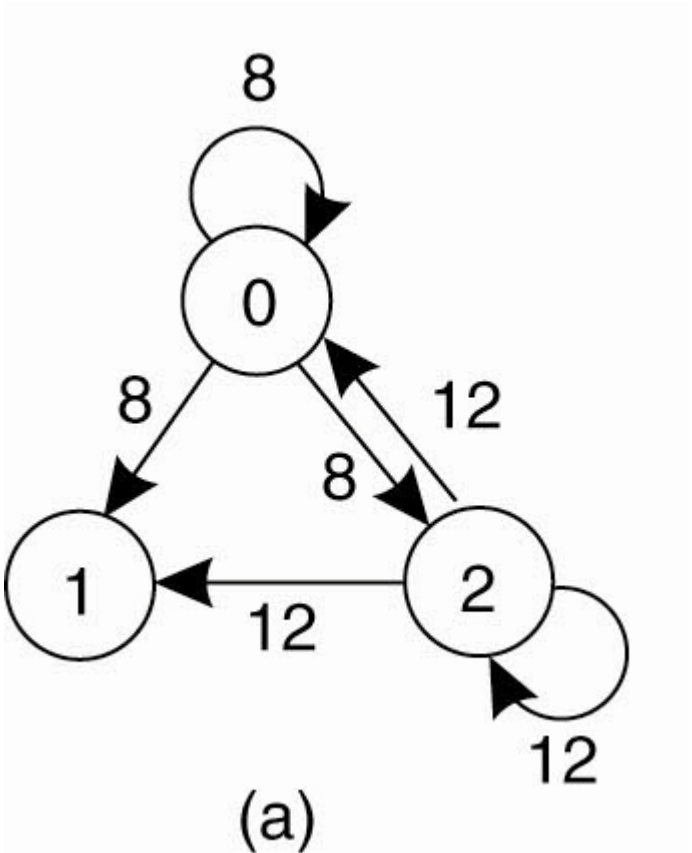
To exit the critical section

state := RELEASED;

reply to any queued requests;

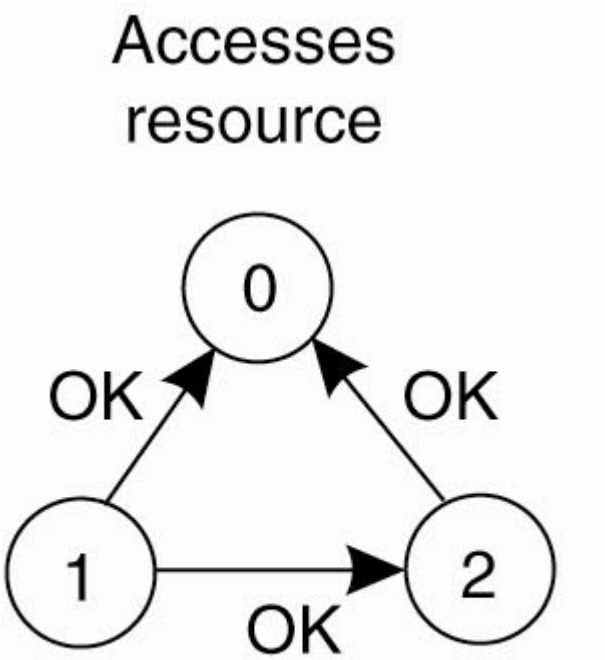
A Distributed Algorithm (2)

- (a) Two processes want to access a shared resource at the same moment.



A Distributed Algorithm (3)

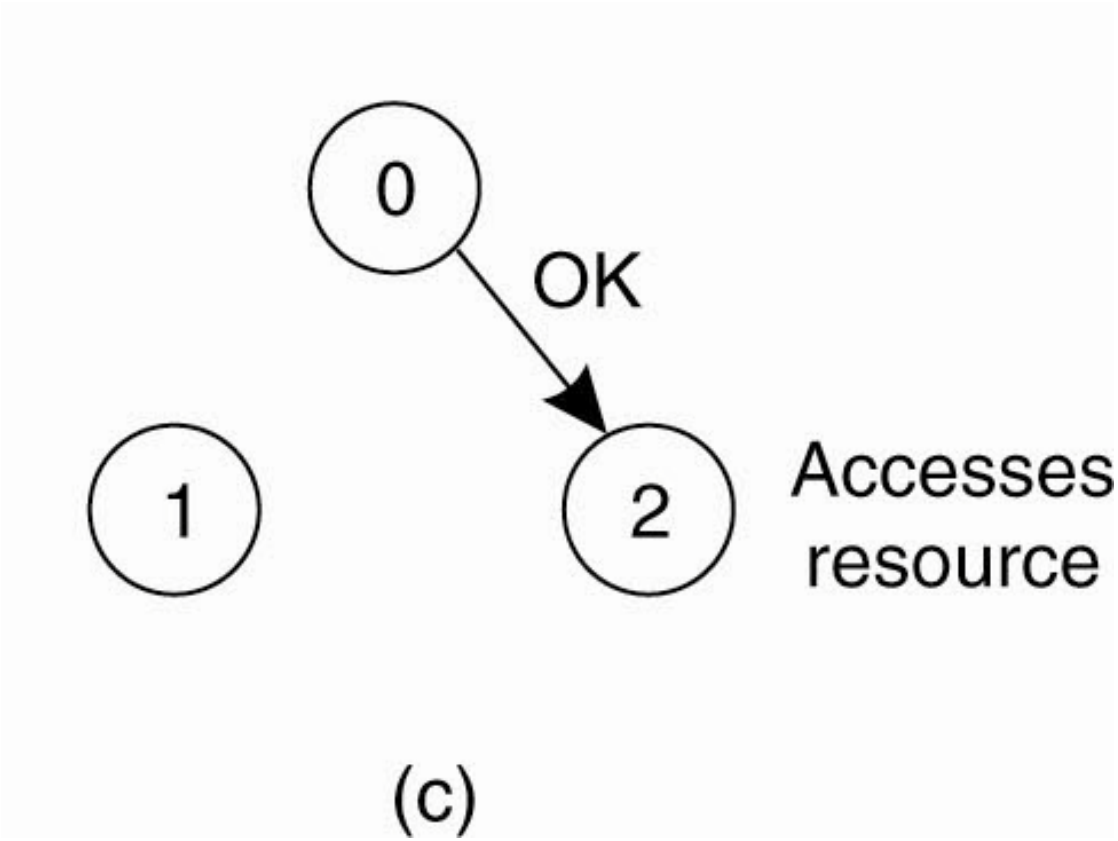
- (b) Process 0 has the lowest timestamp, so it wins.



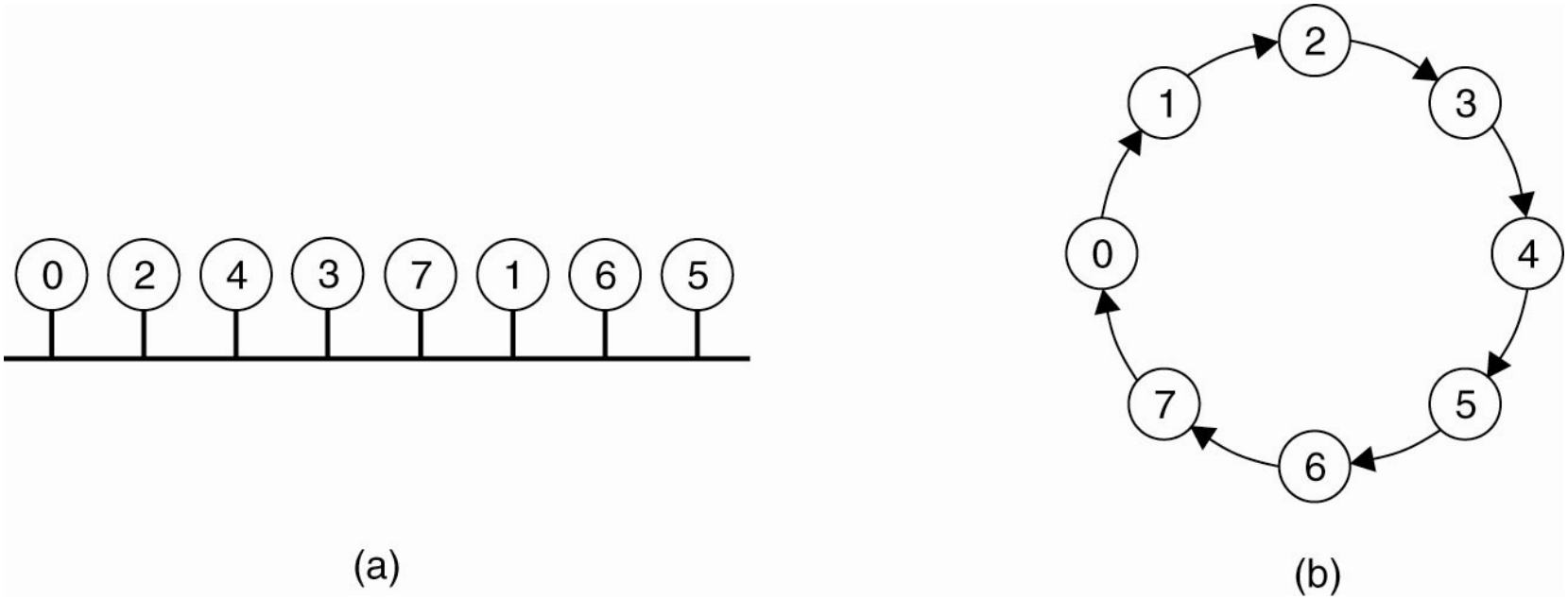
(b)

A Distributed Algorithm (4)

- (c) When process 0 is done,
  it sends an OK also, so 2 can now go ahead.



A Token Ring Algorithm



- (a) An unordered group of processes on a network. (b) A logical ring constructed in software.

A Comparison of the Four Algorithms

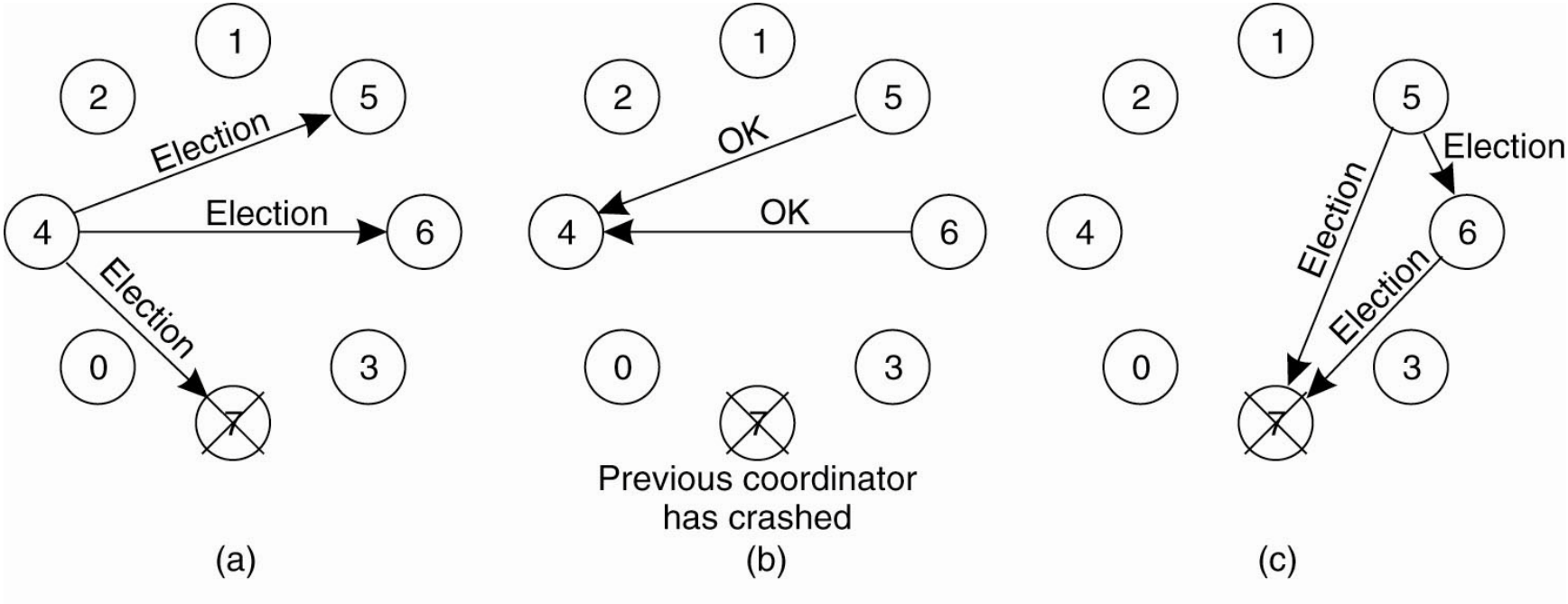
Algorithm	Messages per entry/exit	Delay before entry (in message times)	Problems
Centralized	3	2	Coordinator crash
Decentralized	3mk, k = 1,2,...	2 m	Starvation, low efficiency
Distributed	2 (n – 1)	2 (n – 1)	Crash of any process
Token ring	1 to ∞	0 to n – 1	Lost token, process crash

Election Algorithms

- The Bully Algorithm

1. *P* sends an *ELECTION* message to all processes with higher numbers.
2. If no one responds, *P* wins the election and becomes coordinator.
3. If one of the higher-ups answers, it takes over. *P*'s job is done.

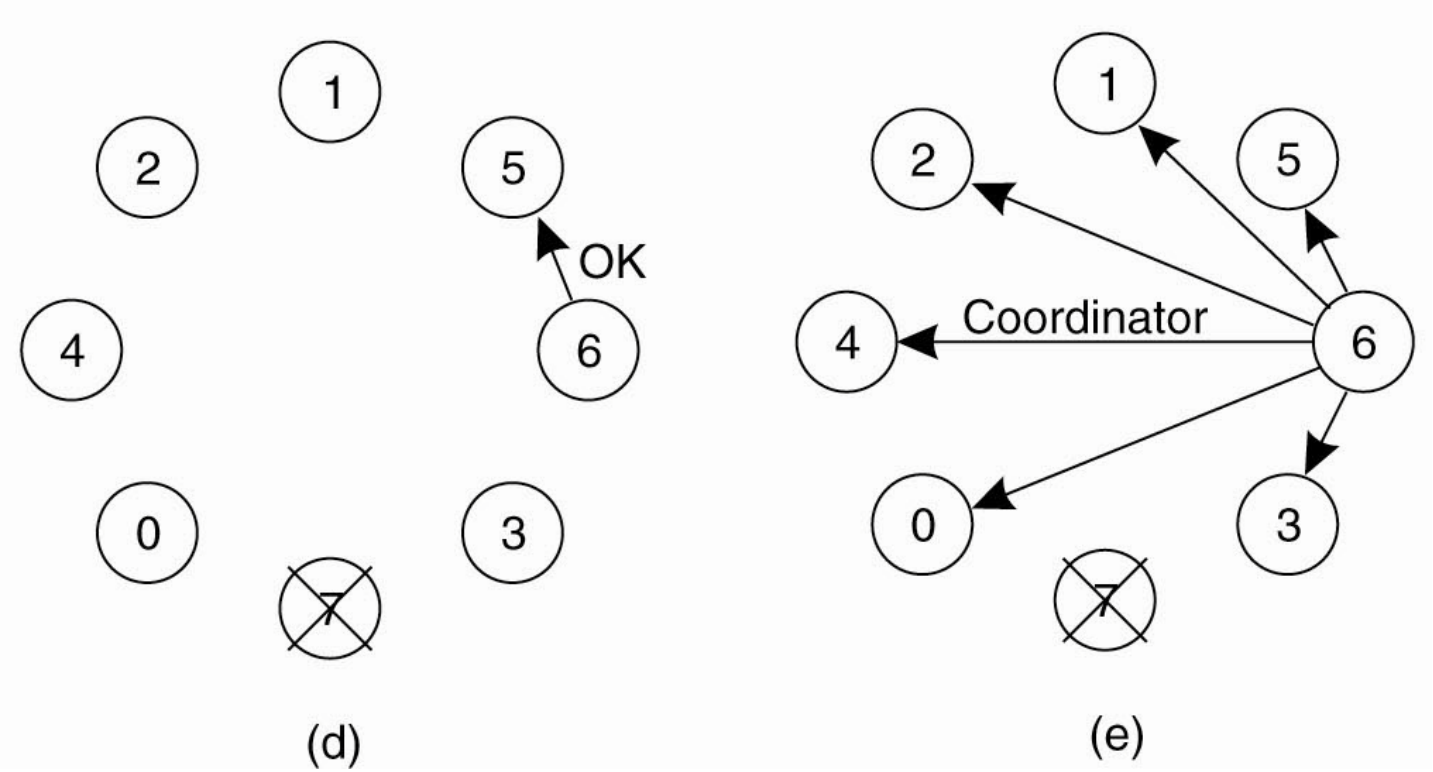
The Bully Algorithm (1)



- The bully election algorithm. (a) Process 4 holds an
- election. (b) Processes 5 and 6 respond, telling 4 to stop.
- (c) Now 5 and 6 each hold an election.

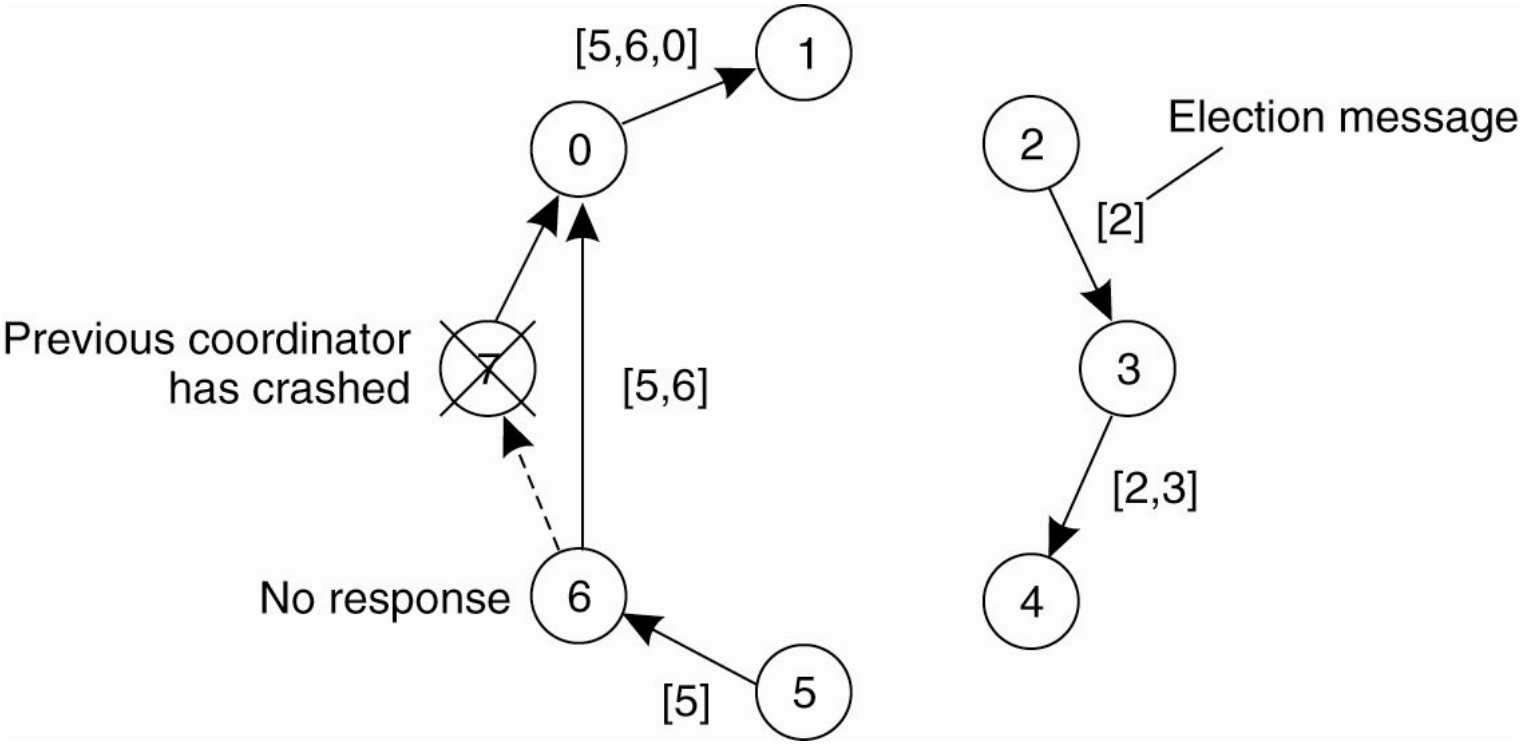
The Bully Algorithm (2)

- The bully election algorithm. (d) Process 6 tells 5 to stop. (e) Process 6 wins and tells everyone.



A Ring Algorithm

- Election algorithm using a ring.



Elections in Large-Scale Systems (1)

- Requirements for superpeer selection:

1. Normal nodes should have low-latency access to superpeers.
2. Superpeers should be evenly distributed across the overlay network.
3. There should be a predefined portion of superpeers relative to the total number of nodes in the overlay network.
4. Each superpeer should not need to serve more than a fixed number of normal nodes.

Reliable multicast algorithm

On initialization

Received := {};

For process p to R-multicast message m to group g

B-multicast(*g*, *m*); // $p \in g$ is included as a destination

On B-deliver(*m*) *at process q with g = group*(*m*)

if ($m \notin$ *Received*)

then

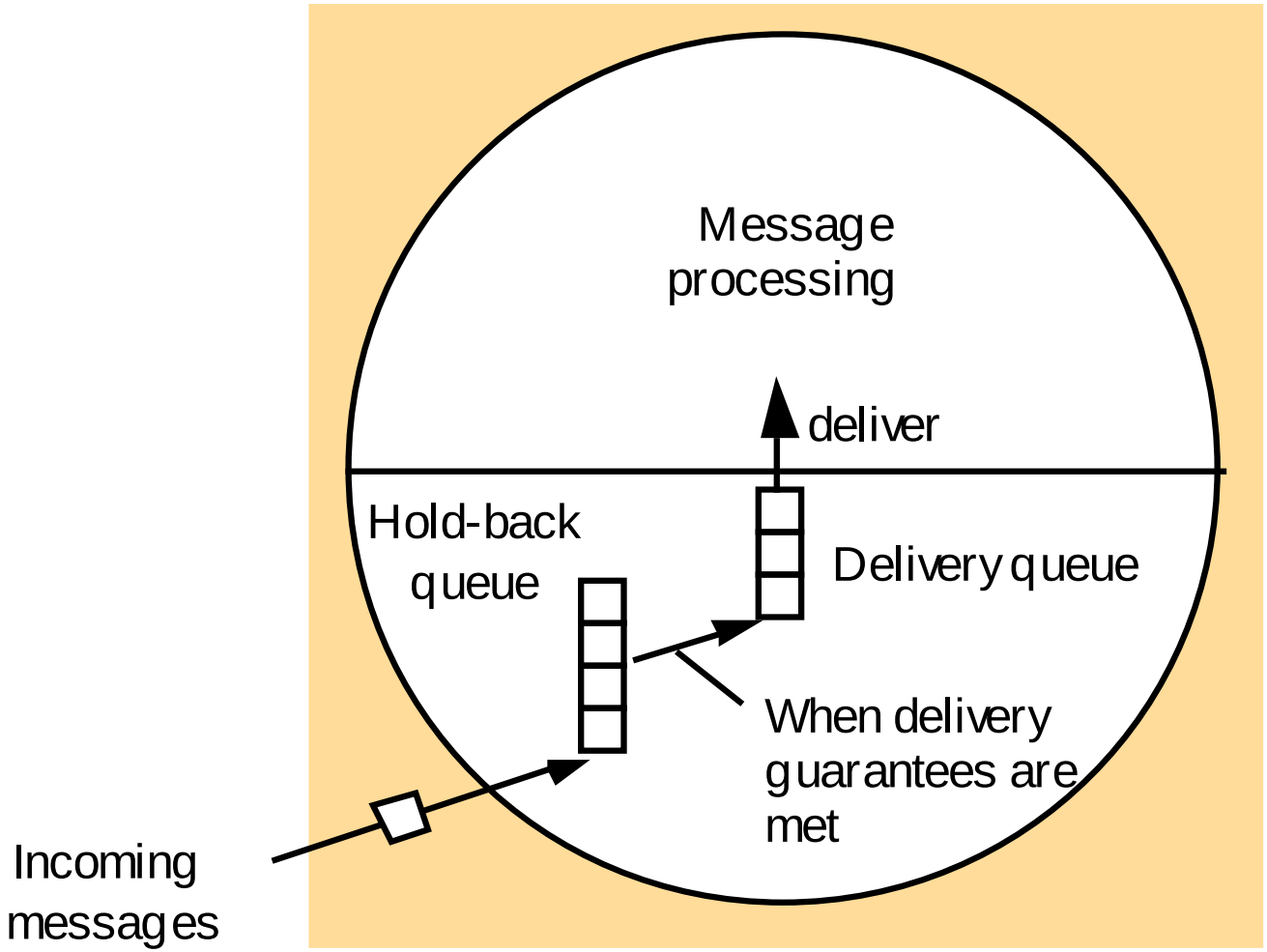
Received := *Received* $\cup$ {*m*};

if ($q \neq p$) *then B-multicast*(*g*, *m*); *end if*

R-deliver m;

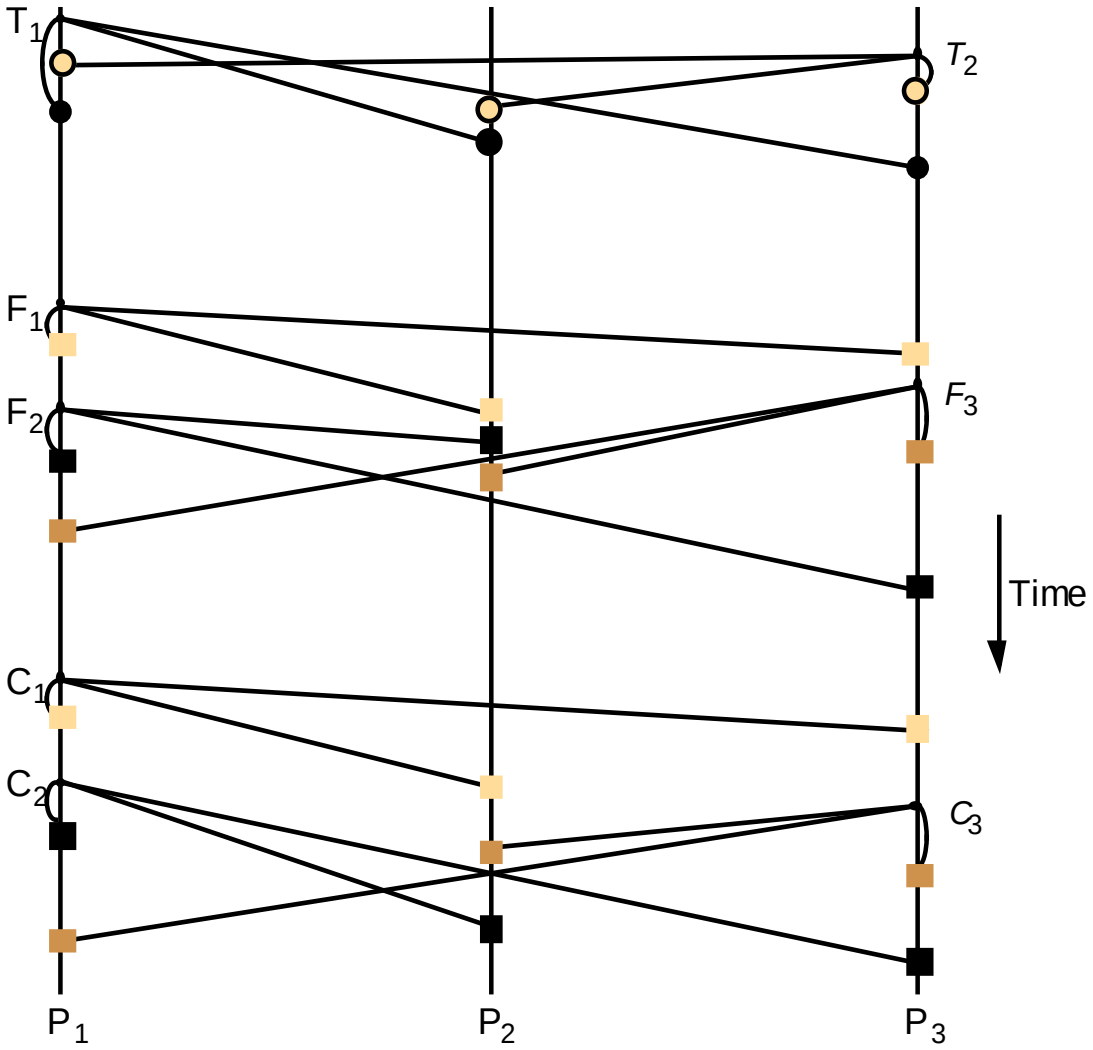
end if

The hold-back queue for arriving multicast messages



Total, FIFO and causal ordering of multicast messages

•Notice the consistent ordering of totally ordered messages T_1 and T_2, the FIFO-related messages F_1 and F_2 and the causally related messages C_1 and C_3 – and the otherwise arbitrary delivery ordering of messages.



Total ordering using a sequencer

1. Algorithm for group member p

On initialization: $r_g := 0$;

To TO-multicast message m to group g
    B-multicast($g \cup \{sequencer(g)\}$, <m, i>);

On B-deliver(<m, i>) *with* $g = group(m)$
    Place <m, i> in hold-back queue;

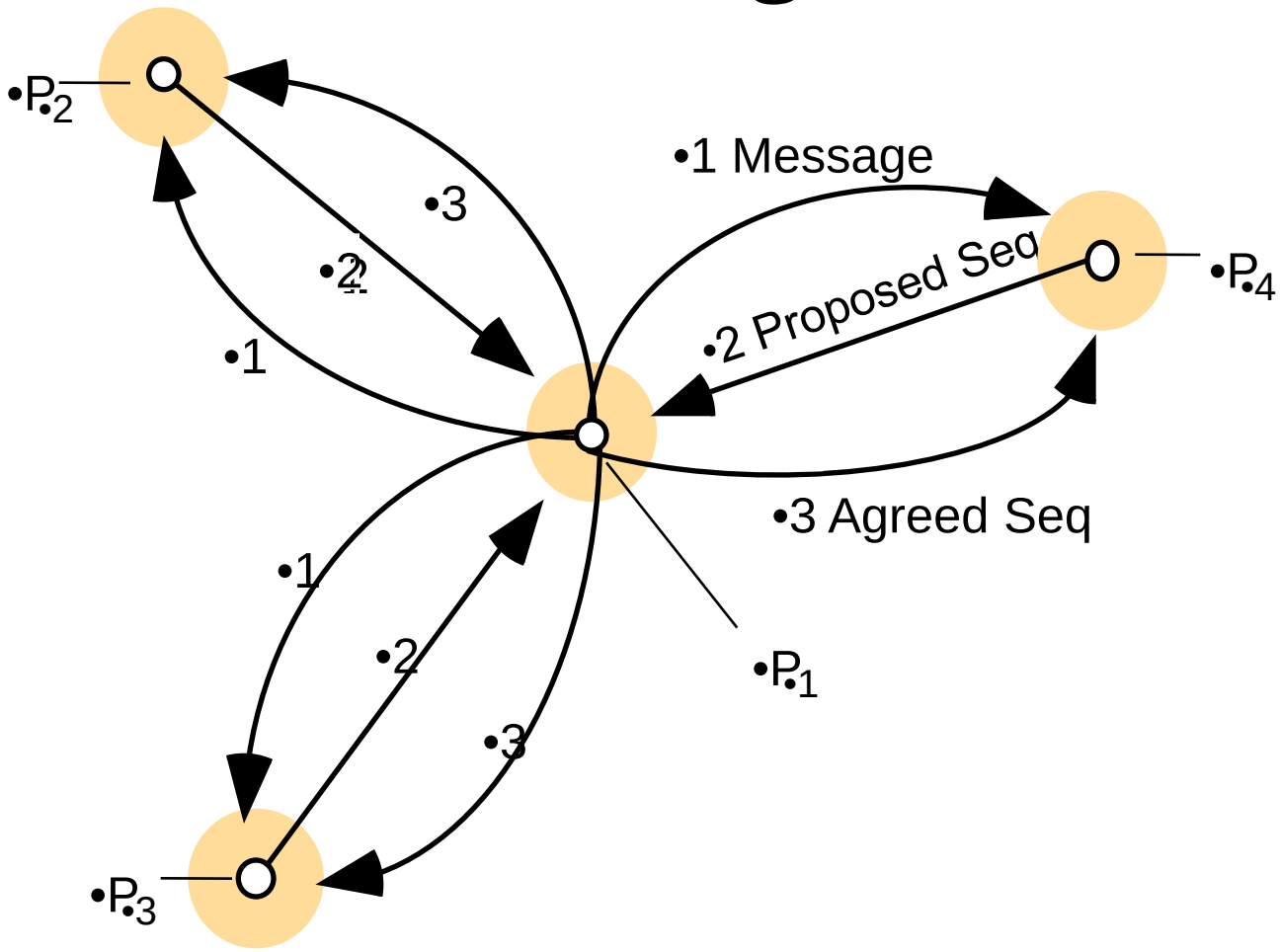
On B-deliver(m_{order} = <"order", i, S>) *with* $g = group(m_{order})$
    wait until <m, i> in hold-back queue and $S = r_g$;
    TO-deliver m; // (after deleting it from the hold-back queue)
    $r_g = S + 1$;

2. Algorithm for sequencer of g

On initialization: $s_g := 0$;

On B-deliver(<m, i>) *with* $g = group(m)$
    B-multicast(g, <"order", i, s_g>);
    $s_g := s_g + 1$;

The ISIS algorithm for total ordering



Causal ordering using vector timestamps

Algorithm for group member p_i ($i = 1, 2\ldots, N$)

On initialization

$V_i^g[j] := 0$ ($j = 1, 2\ldots, N$);

To CO-multicast message m to group g

$V_i^g[i] := V_i^g[i] + 1$;

B-multicast(g, $<V_i^g, m>$);

On B-deliver($<V_j^g, m>$) *from* p_j, *with* $g = group(m)$

place $<V_j^g, m>$ in hold-back queue;

wait until $V_j^g[j] = V_i^g[j] + 1$ and $V_j^g[k] \leq V_i^g[k]$ ($k \neq j$);

CO-deliver m; // after removing it from the hold-back queue

$V_i^g[j] := V_i^g[j] + 1$;